

Disjoint Set Union 2

Last Time

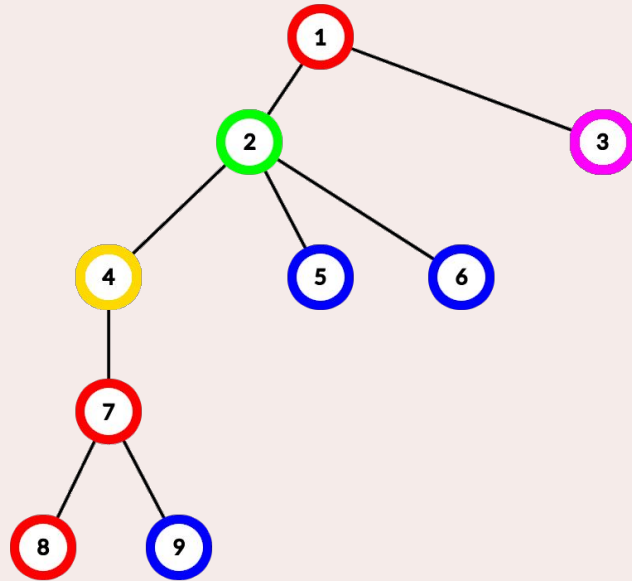
- DSU
 - Path Compression
 - Merge by Size
- Minimum Spanning Tree: Kruskal's

Small-to-Large Merging

Unique Colors in Subtree

You are given a tree consisting of n nodes. Every node has a color.

For each node find the number of distinct colors in the subtree of the node.



Unique Colors in Subtree

Naive solution:

Maintain for each node a list of colors in its subtree. We can merge the lists of colors for the children of a node to get the current list.

Complexity: ?

```
# returns a set of the colors in node
u's subtree
def dfs(u, p):
    colors = {colors[u]}
    for v in adj[u]:
        if v != p:
            child_colors = dfs(v, u)
            for c in child_colors:
                colors.add(c)
    ans[u] = len(colors)
    return colors
```

Unique Colors in Subtree

Naive solution:

Maintain for each node a list of colors in its subtree. We can merge the lists of colors for the children of a node to get the current list.

Complexity: $O(n^2)$

```
# returns a set of the colors in node
u's subtree
def dfs(u, p):
    colors = {colors[u]}
    for v in adj[u]:
        if v != p:
            child_colors = dfs(v, u)
            for c in child_colors:
                colors.add(c)
    ans[u] = len(colors)
    return colors
```

Unique Colors in Subtree

The bottleneck in time complexity comes from merging the sets of colors.

```
# returns a set of the colors in node
u's subtree
def dfs(u, p):
    colors = {colors[u]}
    for v in adj[u]:
        if v != p:
            child_colors = dfs(v, u)
            for c in child_colors:
                colors.add(c)
    ans[u] = len(colors)
    return colors
```

Unique Colors in Subtree

We find the union of two sets a and b (and store it in a) with two ways:

1. Merge b into a in $O(|b|)$
2. Merge a into b in $O(|a|)$

```
# 1) merge b into a
for x in b:
    a.add(x)
```

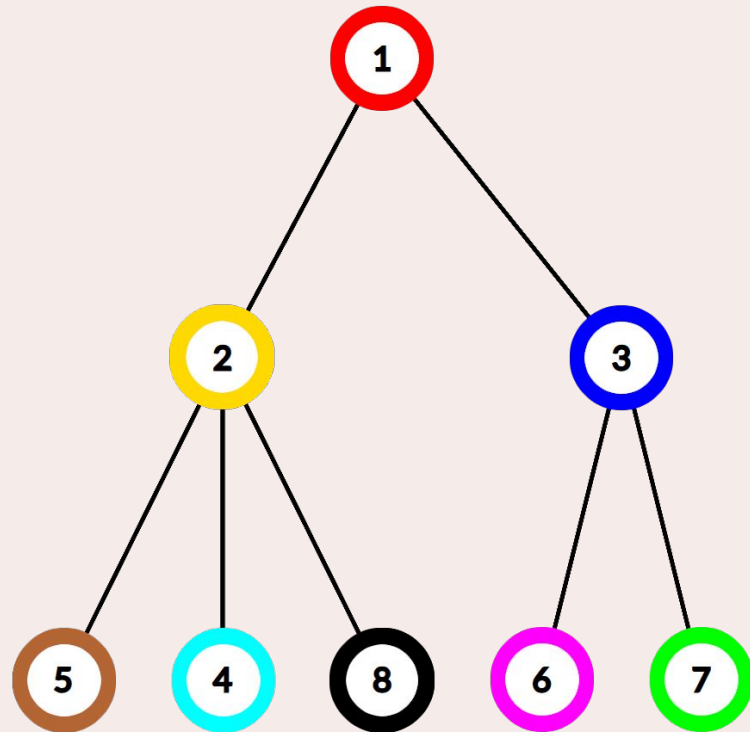
```
# 2) merge a into b
for x in a:
    b.add(x)
a = b
```

Unique Colors in Subtree

Every time a color x “moves lists,” it will join a list with twice x ’s original list’s length.

Consider node 6 (colored magenta):

- $\{6\}$ joins $\{3\}$ to make $\{3, 6\}$
- $\{3, 6, 7\}$ joins $\{1, 2, 5, 4, 8\}$ to make $\{1, 2, 3, 4, 5, 6, 7, 8\}$

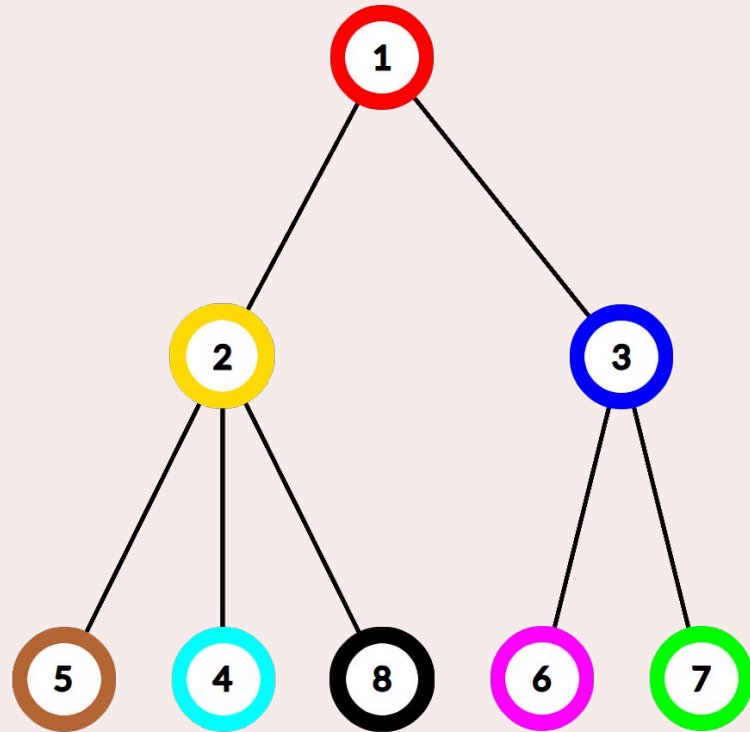


Unique Colors in Subtree

Similar to height analysis in DSU, this means that any individual color moves $O(\log n)$ times.

Summed across all nodes, there are at most $O(n \log n)$ movements.

Final complexity: $O(n \log n)$



Unique Colors in Subtree

```
def dfs(u, p):
    # initialize colors with u's color
    colors = {colors[u]}
    for v in adj[u]:
        if v != p:
            child_colors = dfs(v, u)
            # merge small to large
            if len(colors) > len(child_colors): # merge b into a
                for c in child_colors:
                    colors.add(c)
            else: # merge a into b
                for c in colors:
                    child_colors.add(c)
                colors = child_colors
    ans[u] = len(colors)
    return colors
```

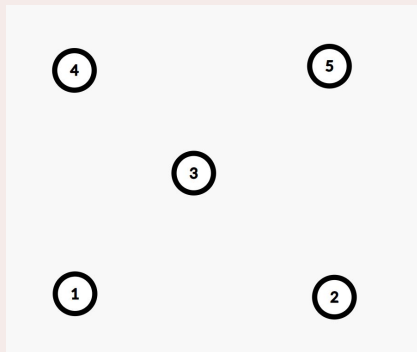
New Roads Queries ([Problem Link](#))

There is a graph of n nodes and initially 0 edges. Over the course of the following m days, one edge will be built every day.

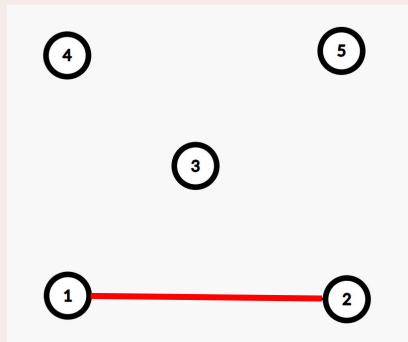
Your task is to process q queries of the form: "after how many days can we travel from node a to node b for the first time?"

Example

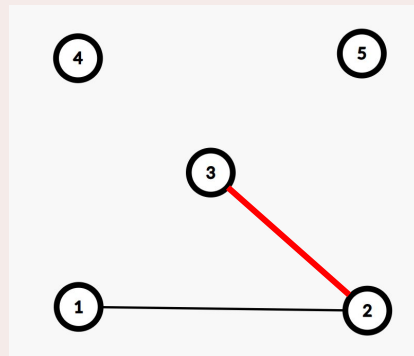
Day 0:



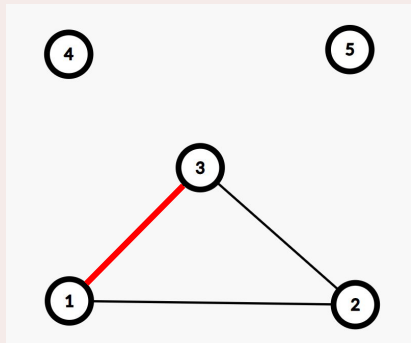
Day 1:



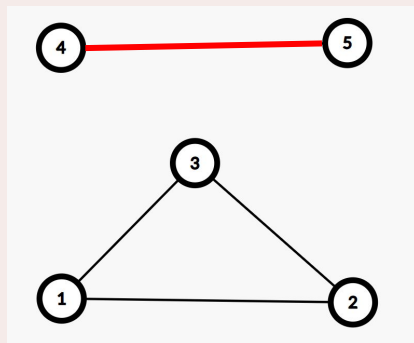
Day 2:



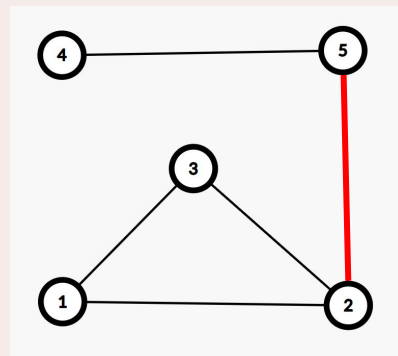
Day 3:



Day 4:

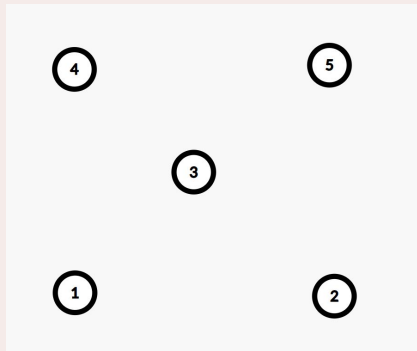


Day 5:

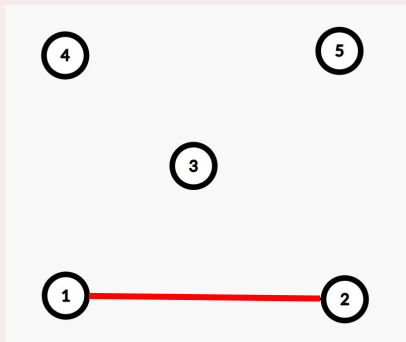


Example: First day 1 and 3 are reachable?

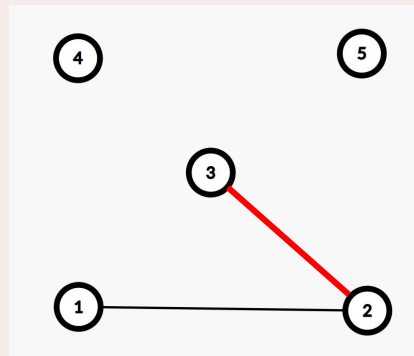
Day 0: ✗



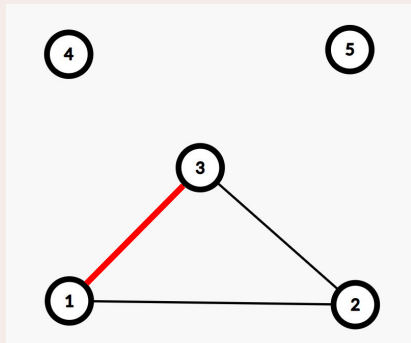
Day 1: ✗



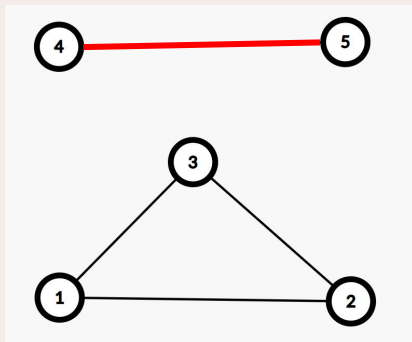
Day 2: ✓



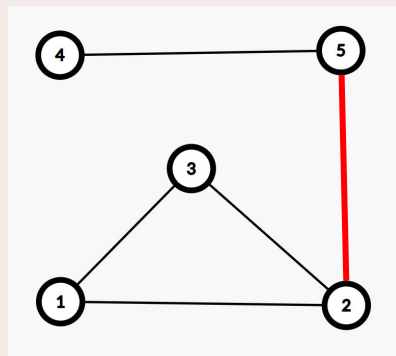
Day 3: ✓



Day 4: ✓

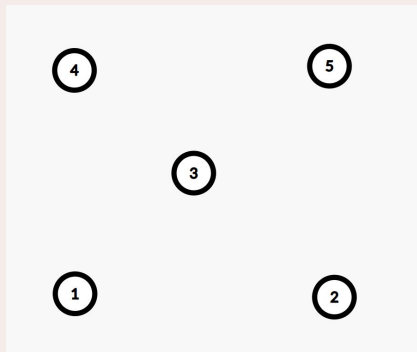


Day 5: ✓

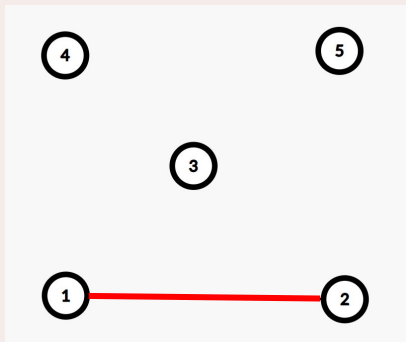


Example: First day 4 and 3 are reachable?

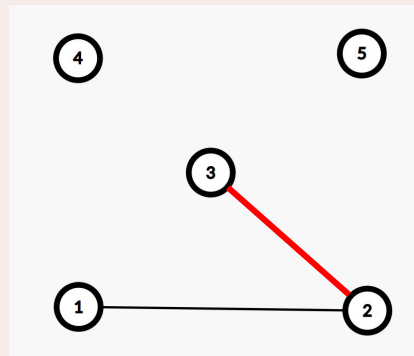
Day 0: ✗



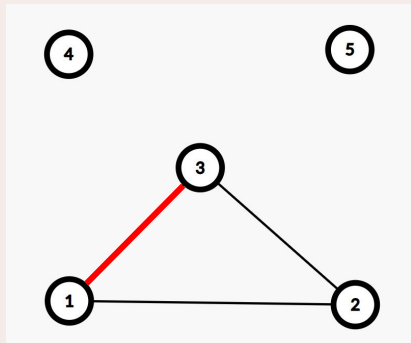
Day 1: ✗



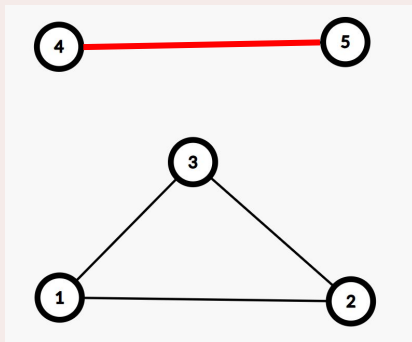
Day 2: ✗



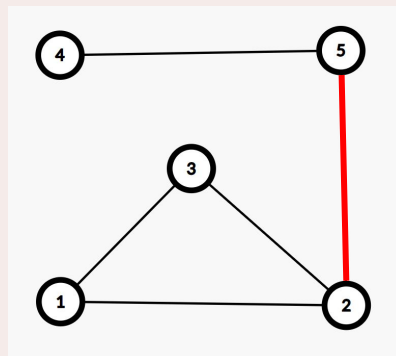
Day 3: ✗



Day 4: ✗

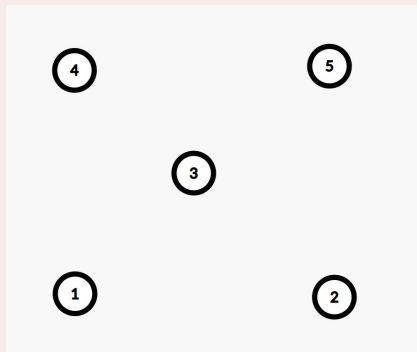


Day 5: ✓

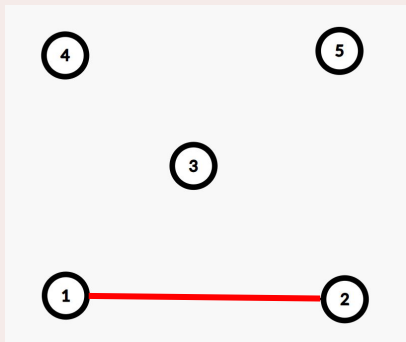


Example: First day 4 and 5 are reachable

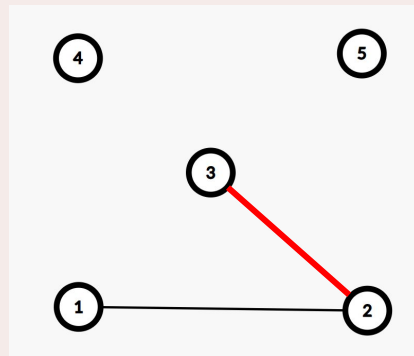
Day 0: ✗



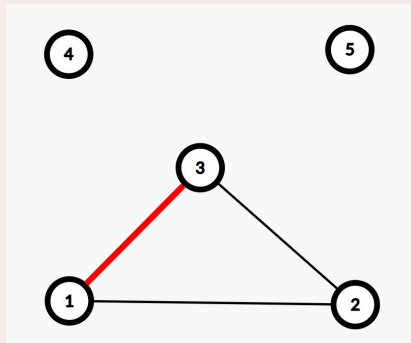
Day 1: ✗



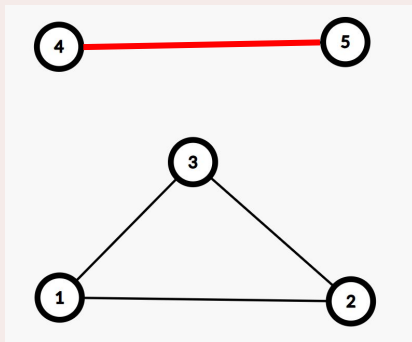
Day 2: ✗



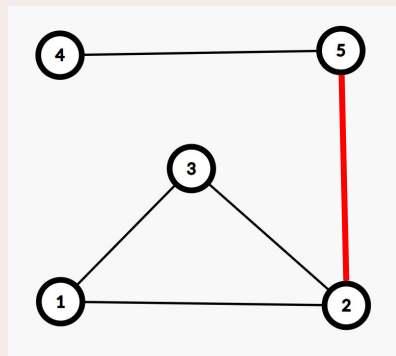
Day 3: ✗



Day 4: ✓

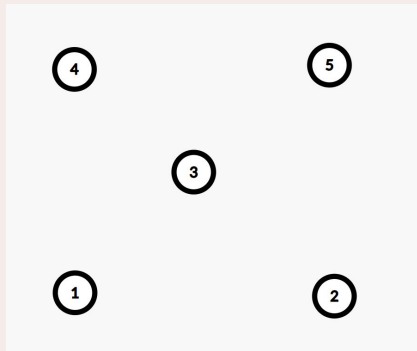


Day 5: ✓

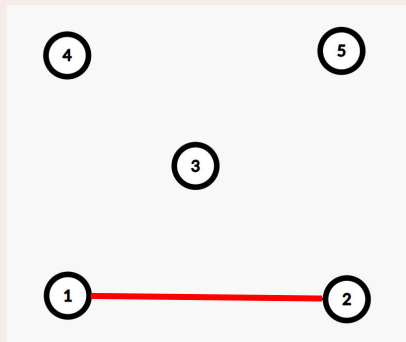


Example: First day 2 and 5 are reachable?

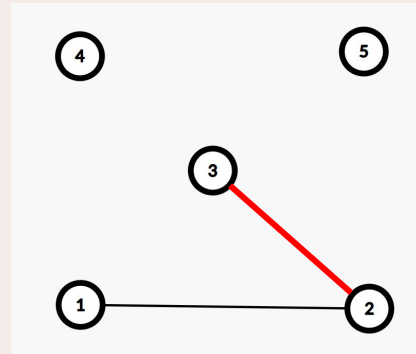
Day 0: ✗



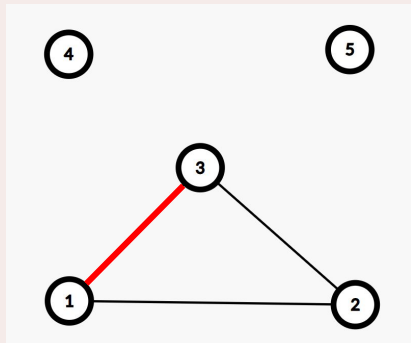
Day 1: ✗



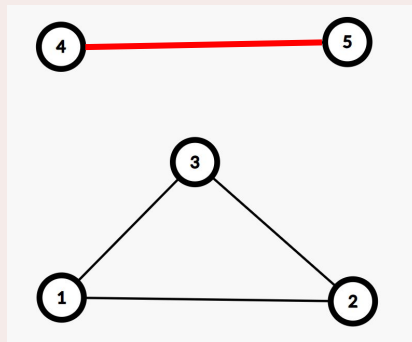
Day 2: ✗



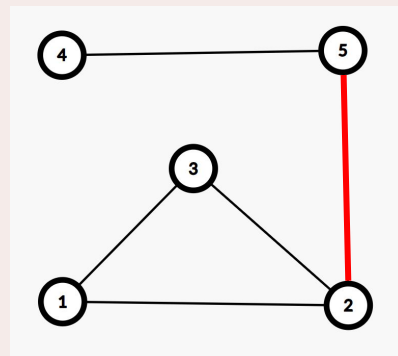
Day 3: ✗



Day 4: ✗

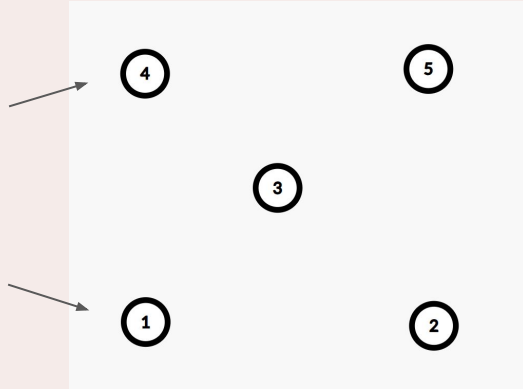


Day 5: ✓

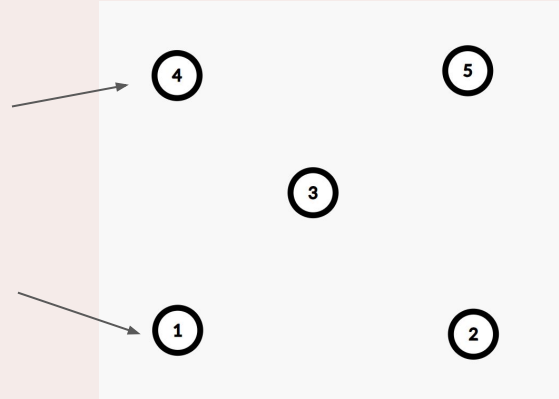


Example Query with $a = 1$, $b = 4$

Day 0:



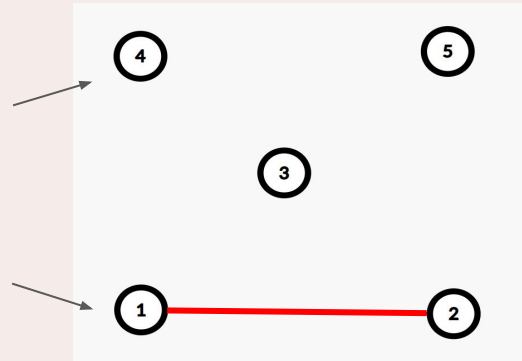
DSU:



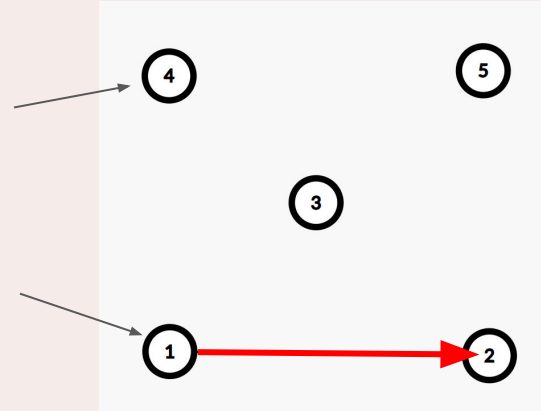
node	1	2	3	4	5
root	1	2	3	4	5
size	1	1	1	1	1

Example Query with $a = 1$, $b = 4$

Day 1:



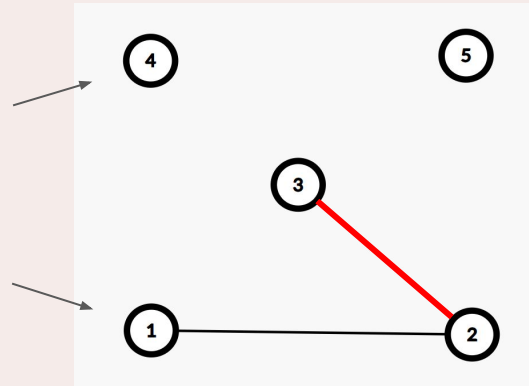
DSU:



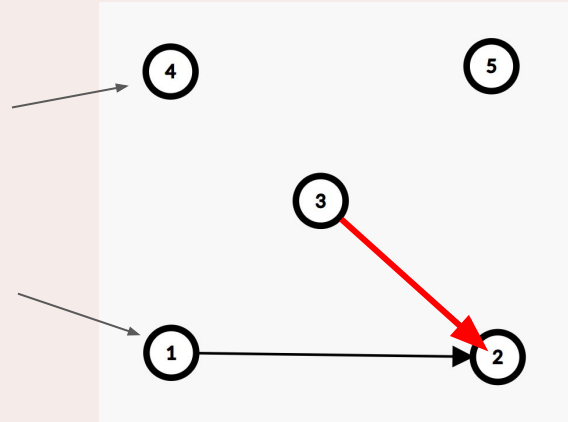
node	1	2	3	4	5
root	2	2	3	4	5
size	N/A	2	1	1	1

Example Query with $a = 1$, $b = 4$

Day 2:



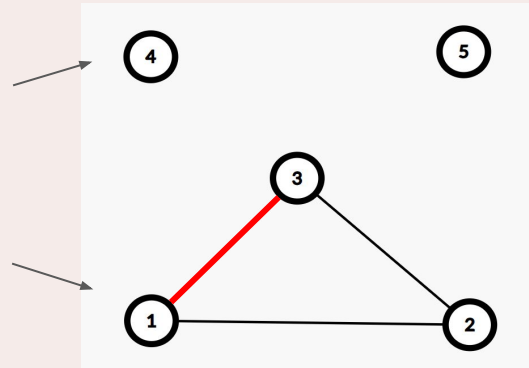
DSU:



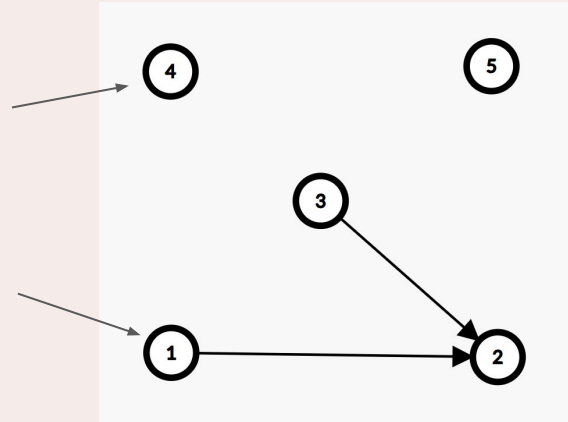
node	1	2	3	4	5
root	2	2	2	4	5
size	N/A	3	N/A	1	1

Example Query with $a = 1$, $b = 4$

Day 3:



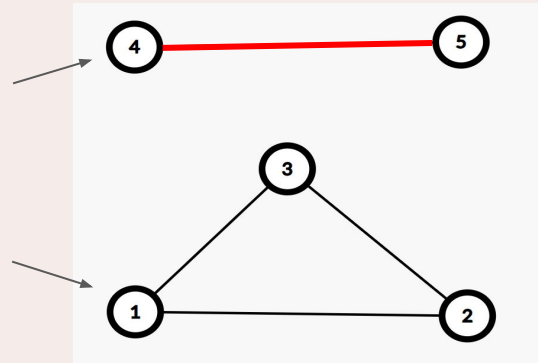
DSU:



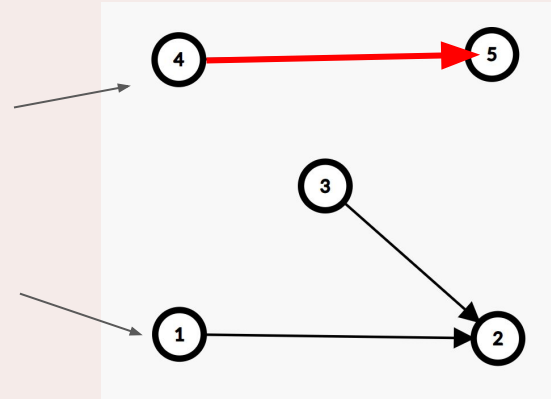
node	1	2	3	4	5
root	2	2	2	4	5
size	N/A	3	N/A	1	1

Example Query with $a = 1$, $b = 4$

Day 4:



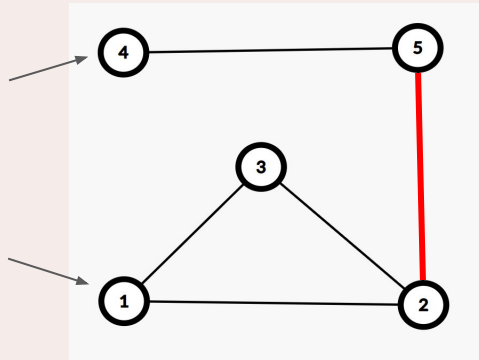
DSU:



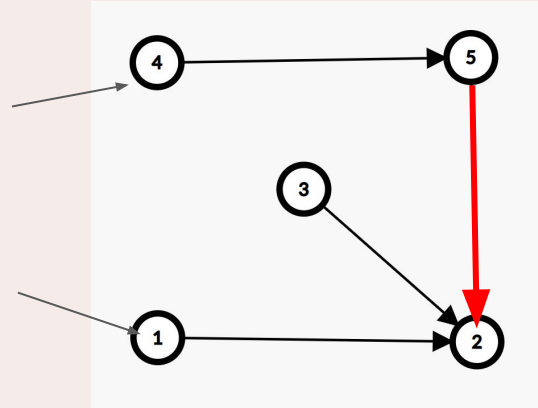
node	1	2	3	4	5
root	2	2	2	5	5
size	N/A	3	N/A	N/A	2

Example Query with $a = 1$, $b = 4$

Day 5:



DSU:

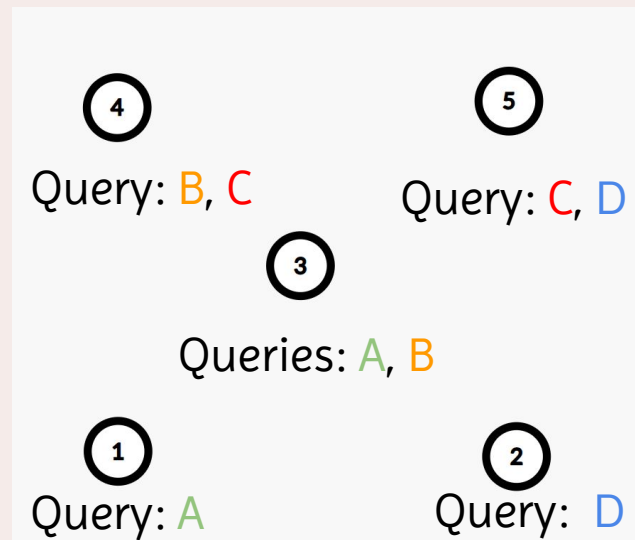


node	1	2	3	4	5
root	2	2	2	2	2
size	N/A	5	N/A	N/A	N/A

New Roads Queries

Initially, split each query (a, b) into two half-queries and put them into nodes a and b.

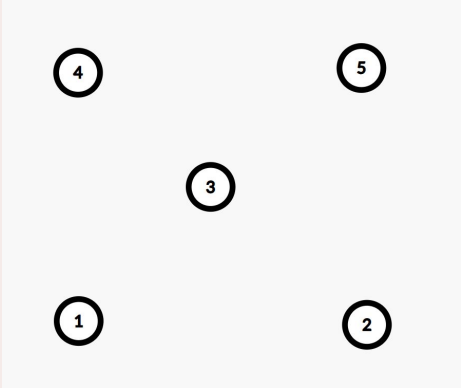
DSU:



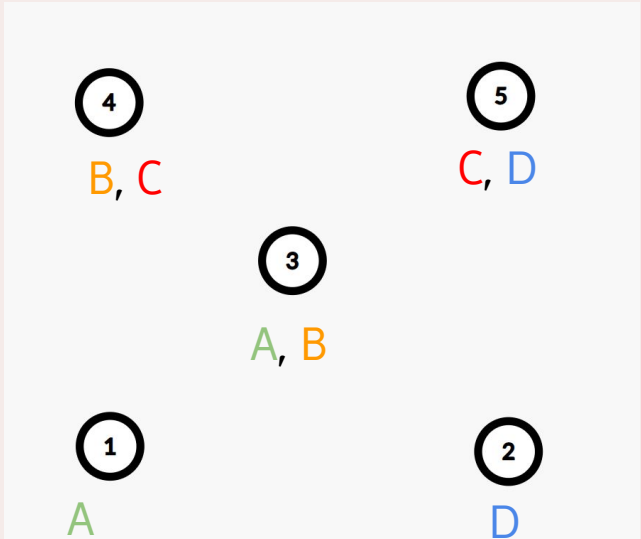
query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	?	?	?	?

New Roads Queries

Day 0:



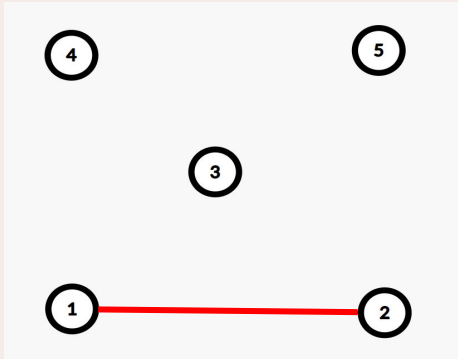
DSU:



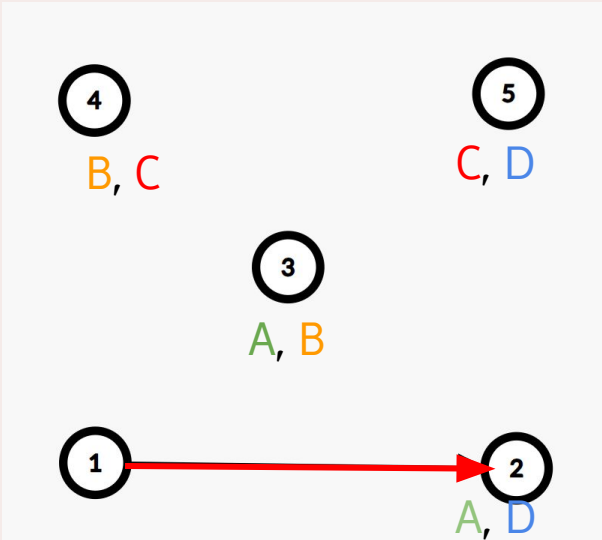
query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	?	?	?	?

New Roads Queries

Day 1:



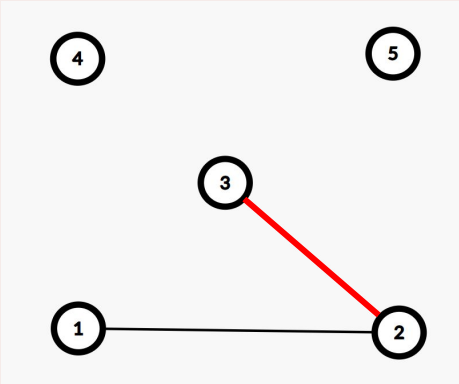
DSU:



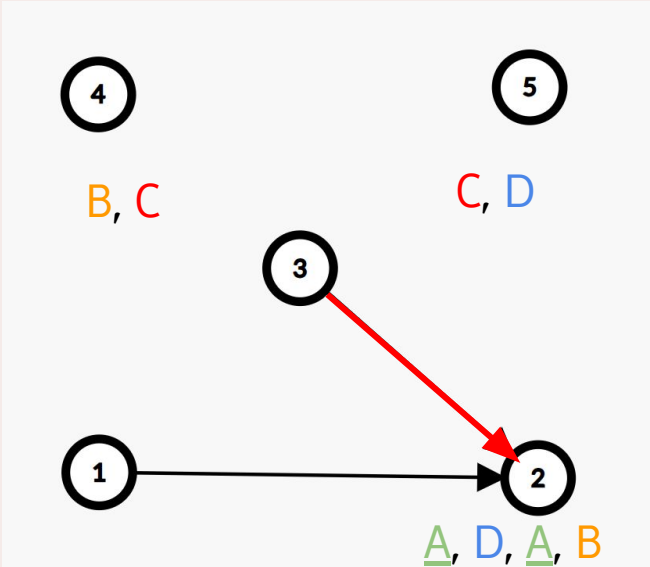
query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	?	?	?	?

New Roads Queries

Day 2:



DSU:

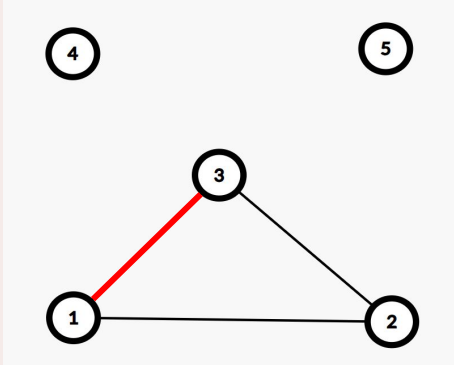


query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	2	?	?	?

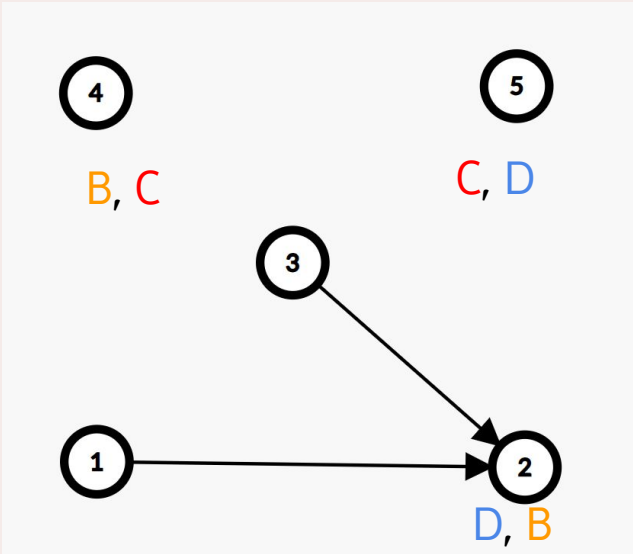
Mit²

New Roads Queries

Day 3:



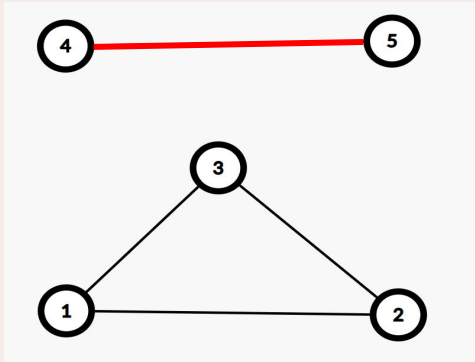
DSU:



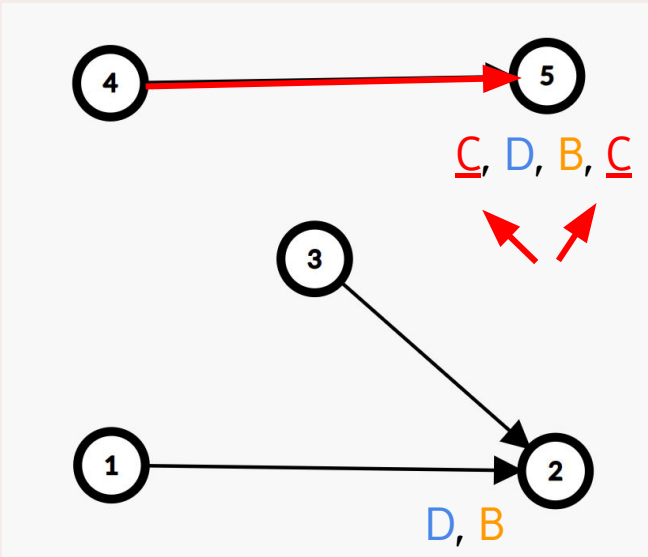
query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	2	?	?	?

New Roads Queries

Day 4:



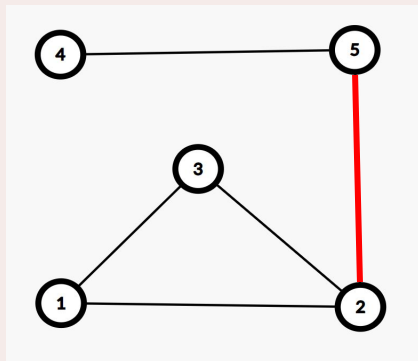
DSU:



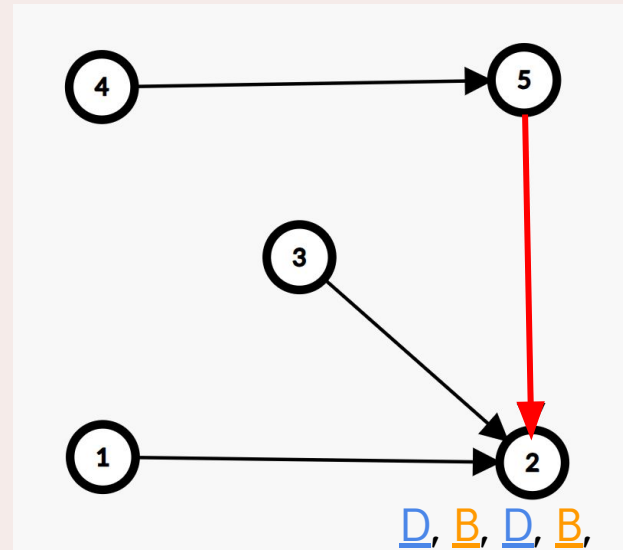
query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	2	?	4	?

New Roads Queries

Day 5:



DSU:



query#	A	B	C	D
(a, b)	(1, 3)	(4, 3)	(4, 5)	(2, 5)
answer	2	5	4	5

New Road Queries

Initially, we will split each query (a, b) into two half-queries and put them into nodes a and b.

We will process the queries by simulating a DSU and “merging lists”.

Finally, we can print the answers.

```
queries = [{} for _ in range(n)]
ans = [-1 for _ in range(q)]
for i in range(q):
    a, b = list(map(int, input().split()))
    queries[a-1].add(i)
    queries[b-1].add(i)

for i in range(m):
    day = i+1
    u, v = edges[i]
    merge(u, v, day)
    # ^ merge function on the next slide

for i in range(q):
    print(ans[i])
```

New Road Queries

We will simulate a DSU, and merge lists of half-queries when merging.

At the time two half-queries of the same query join the same list, we can answer that query.

```
def merge(x: int, y: int, day: int):  
    x = root(x), y = root(y)  
    if x == y:  
        return  
    if sz[x] < sz[y]:  
        x, y = y, x  
    for h in queries[y]:  
        if h in queries[x]:  
            # we have an answer!  
            ans[h] = day  
            queries[x].erase(h)  
        else:  
            queries[x].add(h)  
    sz[x] += sz[y]  
    par[y] = x
```

New Road Queries

We are also doing small-to-large when we union by size!

Every time a query move lists, it joins a new CC with twice the size.

This guarantees $O(n \log n)$ “element movements”.

Final complexity: $O(n \log n)$

```
def merge(x: int, y: int, day: int):  
    x = root(x), y = root(y)  
    if x == y:  
        return  
    if sz[x] < sz[y]:  
        x, y = y, x  
    for h in queries[y]:  
        if h in queries[x]:  
            # we have an answer!  
            ans[h] = day  
            queries[x].erase(h)  
        else:  
            queries[x].add(h)  
    sz[x] += sz[y]  
    par[y] = x
```

DSU With Rollbacks

Sample Problem

You have N nodes, and there are three operations:

1. Merge team A and team B
2. Undo the **last** merge
3. Query whether A and B are on the same team

↶ Undo

Ctrl+Z

Rollback DSU

Without the “rollback,” this is standard DSU

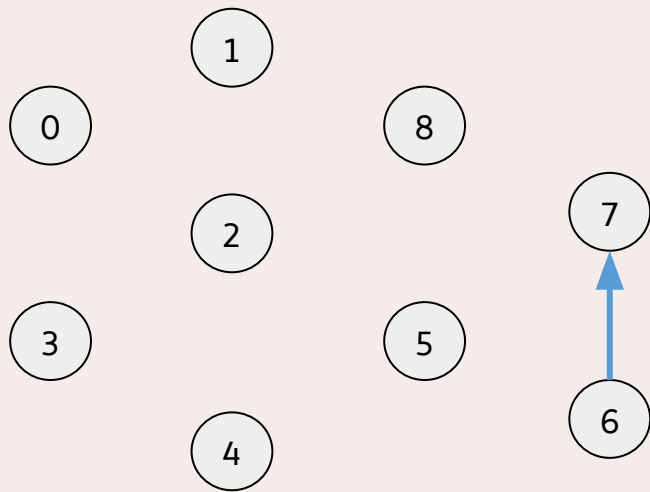
- Keep a stack of the changes that happen
- When we roll back, we reverse the changes that happen

```
def root(x):  
    if par[x] == x:  
        return x  
    return root(par[x])  
  
def join(x, y):  
    x = root(x)  
    y = root(y)  
    if x == y:  
        changes.append(-1)  
        return  
    if sz[x] < sz[y]:  
        x, y = y, x  
    sz[x] += sz[y]  
    changes.append(y)  
    par[y] = x  
  
def rollback():  
    x = changes.pop()  
    if x == -1:  
        return  
    sz[par[x]] -= sz[x]  
    par[x] = x
```

Note: We ONLY use Union by Size. Why?



Rollback DSU



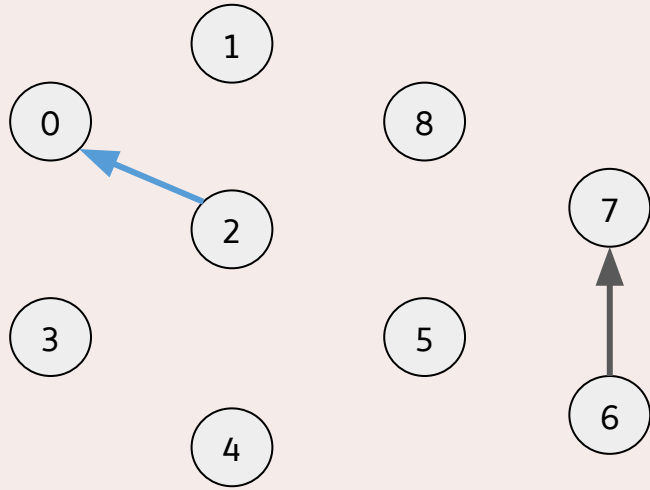
node	0	1	2	3	4	5	6	7	8
par	0	1	2	3	4	5	7	7	8
sz	1	1	1	1	1	1	1	2	1

Stack of Actions in DSU

Top
6

```
def root(x):  
    if par[x] == x:  
        return x  
    return root(par[x])  
  
def join(x, y):  
    x = root(x)  
    y = root(y)  
    if x == y:  
        changes.append(-1)  
        return  
    if sz[x] < sz[y]:  
        x, y = y, x  
    sz[x] += sz[y]  
    changes.append(y)  
    par[y] = x  
  
def rollback():  
    x = changes.pop()  
    if x == -1:  
        return  
    sz[par[x]] -= sz[x]  
    par[x] = x
```

Rollback DSU



Stack of Actions in DSU

Top
2
6

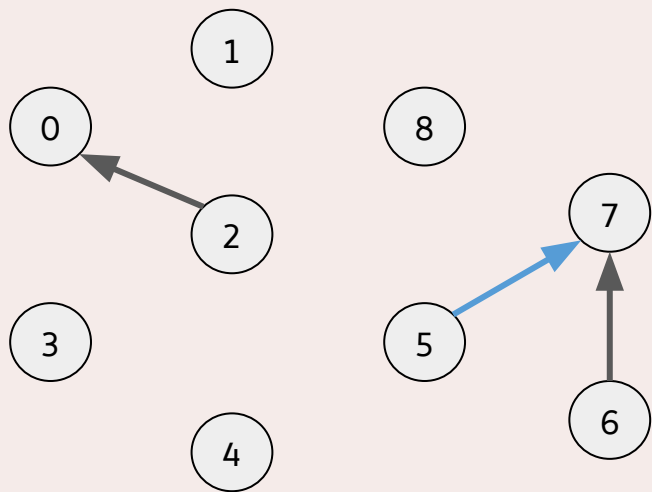
```
def root(x):
    if par[x] == x:
        return x
    return root(par[x])

def join(x, y):
    x = root(x)
    y = root(y)
    if x == y:
        changes.append(-1)
        return
    if sz[x] < sz[y]:
        x, y = y, x
    sz[x] += sz[y]
    changes.append(y)
    par[y] = x

def rollback():
    x = changes.pop()
    if x == -1:
        return
    sz[par[x]] -= sz[x]
    par[x] = x
```

node	0	1	2	3	4	5	6	7	8
par	0	1	0	3	4	5	7	7	8
sz	2	1	1	1	1	1	1	2	1

Rollback DSU



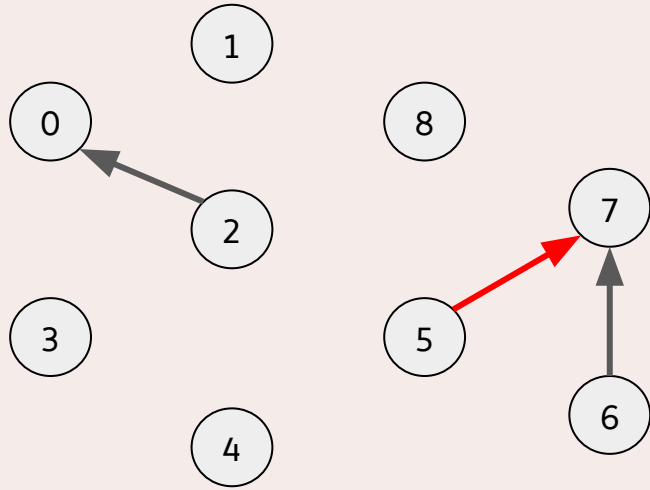
Stack of Actions in DSU

Top
5
2
6

```
def root(x):  
    if par[x] == x:  
        return x  
    return root(par[x])  
  
def join(x, y):  
    x = root(x)  
    y = root(y)  
    if x == y:  
        changes.append(-1)  
        return  
    if sz[x] < sz[y]:  
        x, y = y, x  
    sz[x] += sz[y]  
    changes.append(y)  
    par[y] = x  
  
def rollback():  
    x = changes.pop()  
    if x == -1:  
        return  
    sz[par[x]] -= sz[x]  
    par[x] = x
```

node	0	1	2	3	4	5	6	7	8
par	0	1	0	3	4	7	7	7	8
sz	2	1	1	1	1	1	1	3	1

Rollback DSU



Stack of Actions in DSU

Top
2
6

```

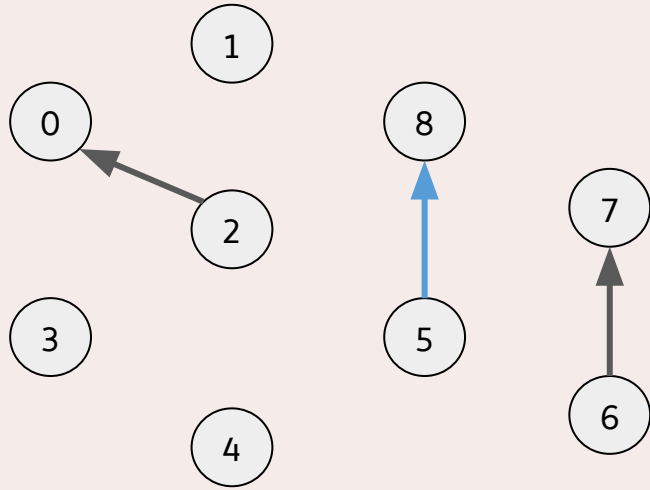
def root(x):
    if par[x] == x:
        return x
    return root(par[x])

def join(x, y):
    x = root(x)
    y = root(y)
    if x == y:
        changes.append(-1)
        return
    if sz[x] < sz[y]:
        x, y = y, x
    sz[x] += sz[y]
    changes.append(y)
    par[y] = x

def rollback():
    x = changes.pop()
    if x == -1:
        return
    sz[par[x]] -= sz[x]
    par[x] = x
  
```

node	0	1	2	3	4	5	6	7	8
par	0	1	0	3	4	5	7	7	8
sz	2	1	1	1	1	1	1	2	1

Rollback DSU



Stack of Actions in DSU

Top
5
2
6

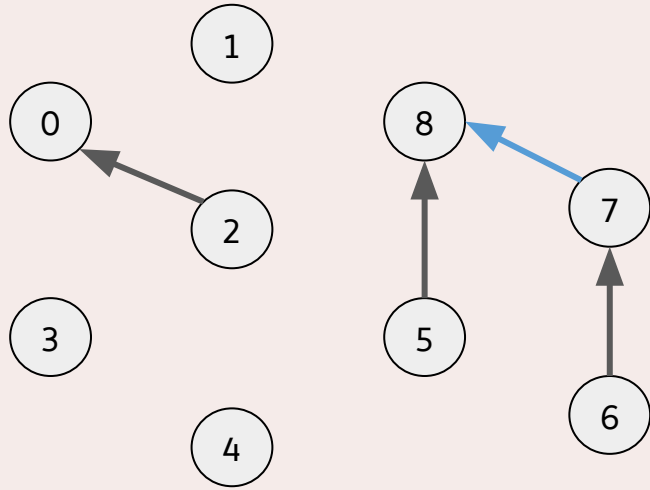
```
def root(x):
    if par[x] == x:
        return x
    return root(par[x])

def join(x, y):
    x = root(x)
    y = root(y)
    if x == y:
        changes.append(-1)
        return
    if sz[x] < sz[y]:
        x, y = y, x
    sz[x] += sz[y]
    changes.append(y)
    par[y] = x

def rollback():
    x = changes.pop()
    if x == -1:
        return
    sz[par[x]] -= sz[x]
    par[x] = x
```

node	0	1	2	3	4	5	6	7	8
par	0	1	0	3	4	8	7	7	8
sz	2	1	1	1	1	1	1	2	2

Rollback DSU



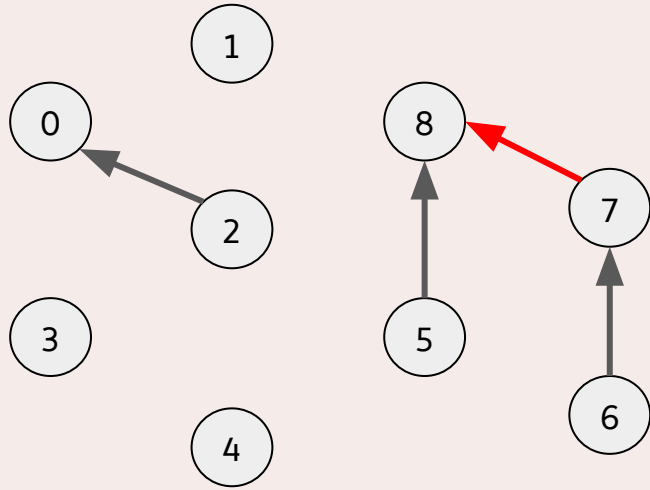
Stack of Actions in DSU

Top
7
5
2
6

```
def root(x):  
    if par[x] == x:  
        return x  
    return root(par[x])  
  
def join(x, y):  
    x = root(x)  
    y = root(y)  
    if x == y:  
        changes.append(-1)  
        return  
    if sz[x] < sz[y]:  
        x, y = y, x  
    sz[x] += sz[y]  
    changes.append(y)  
    par[y] = x  
  
def rollback():  
    x = changes.pop()  
    if x == -1:  
        return  
    sz[par[x]] -= sz[x]  
    par[x] = x
```

node	0	1	2	3	4	5	6	7	8
par	0	1	0	3	4	8	7	8	8
sz	2	1	1	1	1	1	1	2	4

Rollback DSU



Stack of Actions in DSU

Top
5
2
6

```

def root(x):
    if par[x] == x:
        return x
    return root(par[x])

def join(x, y):
    x = root(x)
    y = root(y)
    if x == y:
        changes.append(-1)
        return
    if sz[x] < sz[y]:
        x, y = y, x
    sz[x] += sz[y]
    changes.append(y)
    par[y] = x

def rollback():
    x = changes.pop()
    if x == -1:
        return
    sz[par[x]] -= sz[x]
    par[x] = x
  
```

node	0	1	2	3	4	5	6	7	8
par	0	1	0	3	4	8	7	7	8
sz	2	1	1	1	1	1	1	2	2

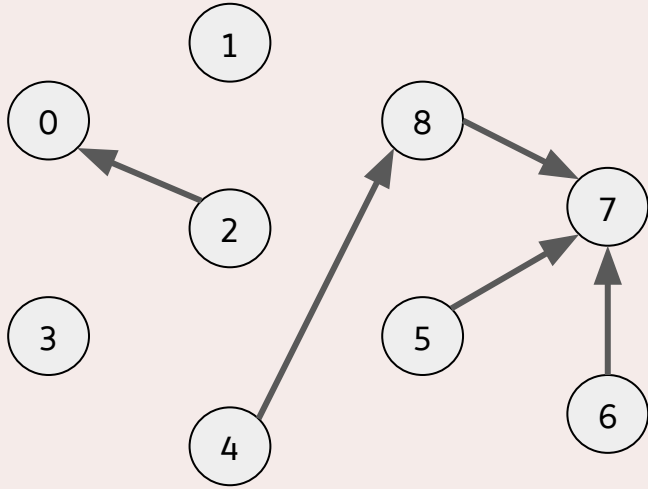
Dynamic Connectivity

Sample Problem

You have N nodes, and there are three operations:

1. Add an edge between node A and node B
2. Remove an edge between node A and node B
3. Query whether A and B are on the same connected component

Naïve Approach (DSU with Rollback)

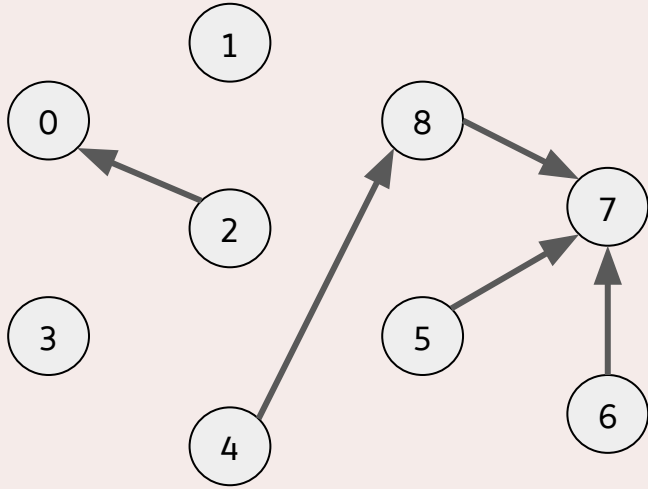


Stack of Actions in DSU

Top
8
4
5
2
6

Node	0	1	2	3	4	5	6	7	8
Par	0	1	0	3	8	7	7	7	7

Naïve Approach (DSU with Rollback)



Stack of Actions in DSU

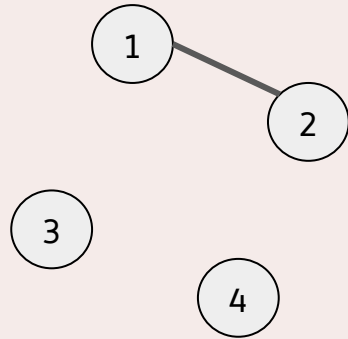
Top
8
4
5
2
6

How do we delete 6 → 7?

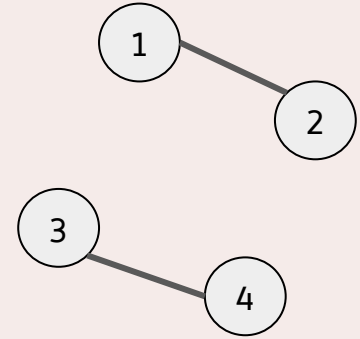
Node	0	1	2	3	4	5	6	7	8
Par	0	1	0	3	8	7	7	7	7

Sample Graph

Time 2:



Time 8:



Query: at $t = 2$, are 2 and 3 connected?



Query: at $t = 8$, are 3 and 4 connected?

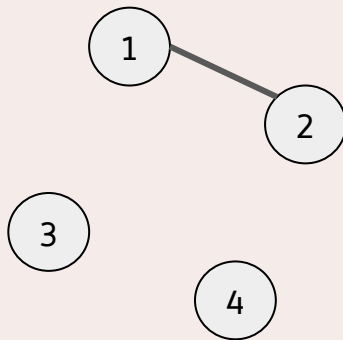


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 1:

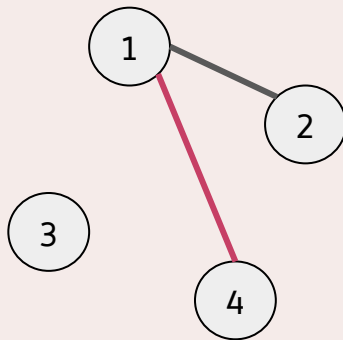


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 2:

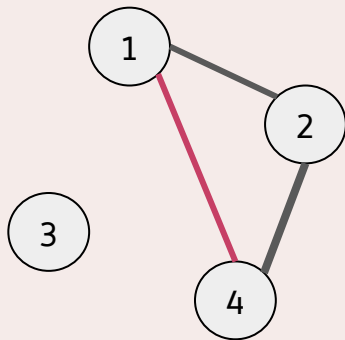


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 3:

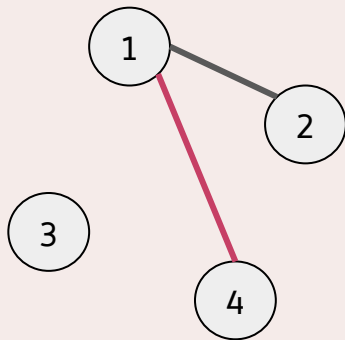


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 4:

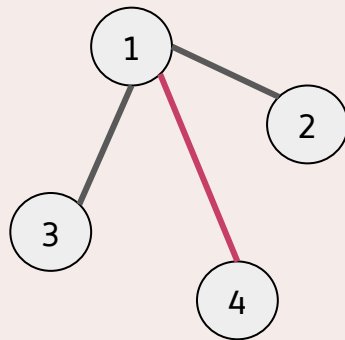


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 5:

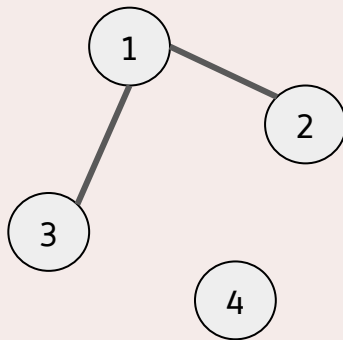


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 6:

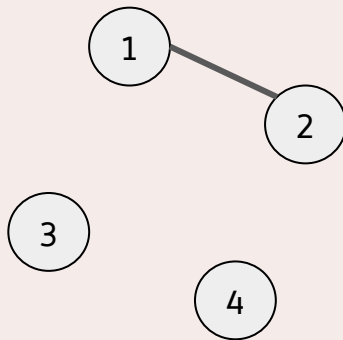


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 7:

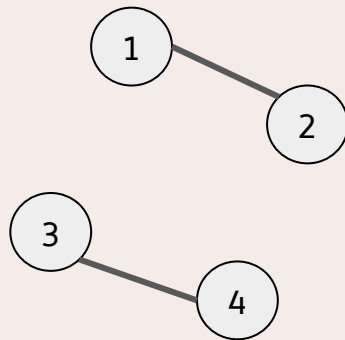


Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Reduction to Range Query

Key Observation: **Any edge is active in the graph for an interval: [insertion, deletion)**

Time 8:



Time	1	2	3	4	5	6	7	8
Type	Insert	Insert	Insert	Delete	Insert	Delete	Delete	Insert
Edge	$1 \leftrightarrow 2$	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$	$2 \leftrightarrow 4$	$1 \leftrightarrow 3$	$1 \leftrightarrow 4$	$1 \leftrightarrow 3$	$3 \leftrightarrow 4$

Segment Tree

When an edge is present for a range, we add the edge to all of its corresponding ranges.

Insert $1 \leftrightarrow 4$ along this range:

1	2	3	4	5	6	7	8

Segment Tree

When an edge is present for a range, we add the edge to all of its corresponding ranges.

Insert $1 \leftrightarrow 3$

		$1 \leftrightarrow 4$					
	$1 \leftrightarrow 4$			$1 \leftrightarrow 4$			
1	2	3	4	5	6	7	8

Dynamic Connectivity

Maintain a rollback DSU, which will find our answer.

Suppose we're currently at a node.

We add all of the edges we find at that node to our DSU, then roll back the changes once we leave it.

Once we reach a leaf node, we answer all of the queries at that time stamp

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

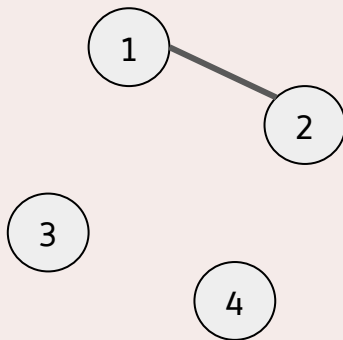
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$t = 1$:



$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$	$1 \leftrightarrow 3$				
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```

def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
  
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

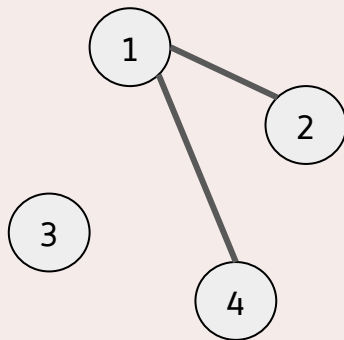
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$t = 2$:



At $t = 2$, are 2 and 3 connected?

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```

def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
  
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$	$1 \leftrightarrow 3$				
$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$	

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

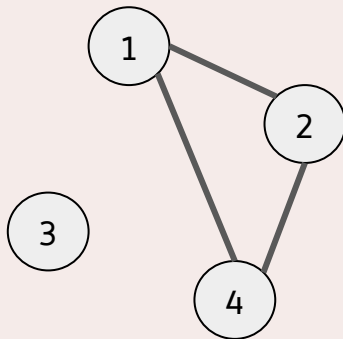
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $2 \leftrightarrow 4$
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$t = 3$:



1 ↔ 2						
		1 ↔ 4	1 ↔ 3			
1 ↔ 4	2 ↔ 4		1 ↔ 4			3 ↔ 4

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$	$1 \leftrightarrow 3$				
$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$				$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

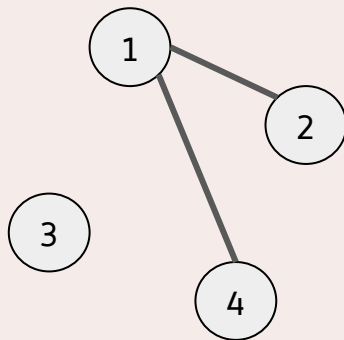
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$t = 2:$



$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$	$1 \leftrightarrow 3$				
$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$				$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 3$
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

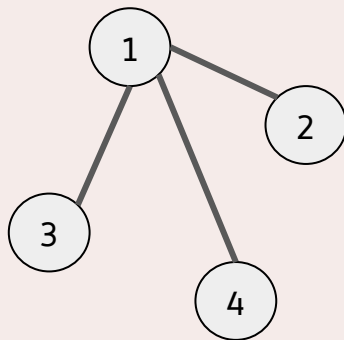
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 4$
 $1 \leftrightarrow 3$
 $1 \leftrightarrow 2$

$t = 5:$



$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```

def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
  
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 3$
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$	$1 \leftrightarrow 3$				
$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$				$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

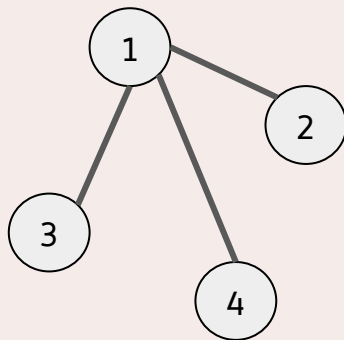
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 3$
 $1 \leftrightarrow 2$

$t = 6:$



$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 3$
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$	$1 \leftrightarrow 3$				
$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$				$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

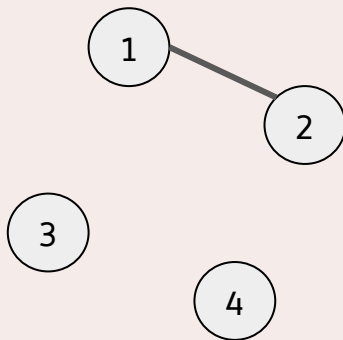
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$t = 7:$



1 ↔ 2							
		1 ↔ 4		1 ↔ 3			
	1 ↔ 4	2 ↔ 4		1 ↔ 4			3 ↔ 4

```

def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
  
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

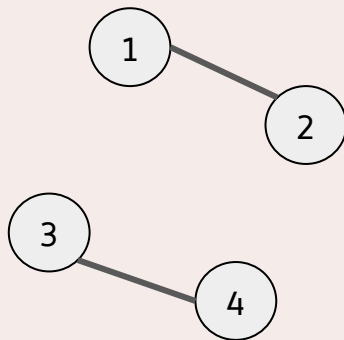
def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $3 \leftrightarrow 4$
 $1 \leftrightarrow 2$

$t = 8$:



At $t = 8$, are 3 and 4 connected?

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```

def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
  
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Dynamic Connectivity

Stack of Actions in DSU

Top
 $1 \leftrightarrow 2$

$1 \leftrightarrow 2$							
		$1 \leftrightarrow 4$		$1 \leftrightarrow 3$			
	$1 \leftrightarrow 4$	$2 \leftrightarrow 4$		$1 \leftrightarrow 4$			$3 \leftrightarrow 4$

```
def insert(i, l, r, ql, qr, edge):
    if ql <= l and r <= qr:
        edges[i].append(edge)
        return
    if r < ql or qr < l:
        return
    mid = (l + r) // 2
    insert(i * 2, l, mid, ql, qr, edge)
    insert(i * 2 + 1, mid + 1, r, ql, qr, edge)
    return

def solve(i, l, r):
    for a, b in edges[i]:
        join(a, b)
    if l == r:
        # do queries
        pass
    else:
        mid = (l + r) // 2
        solve(i * 2, l, mid)
        solve(i * 2 + 1, mid + 1, r)
    for a, b in edges[i]:
        rollback()
```

Thanks for Coming!

- Rollback DSU

*Attendance Form below!
(1 raffle ticket)*

[Raffle Counting
Form](#) (for last week,
last call)

